

Accelerating integer C code using FPU in parallel

Presented by

T. Amrutha CS24BTECH11060

M. Reethika CS24BTECH11038

Hardware overview (ARM Cortex A-76)

Execution units:

- Integer ALUs
- FP/NEON unit
- Load store unit
- Branch execution unit
- In case of auto vectorization FP/NEON unit is not idle since vectorization is being done in FP unit

Reference : Technical reference manual for ARM Cortex A-76.

- O3 optimization enables aggressive auto vectorization
- There is one instruction used in our c codes i.e fma instruction which does floating point multiplication and addition in a single cycle.

The Cortex-A76 core includes the following features:

Core features

- 40-bit *Physical Address* (PA).
- A *Memory Management Unit* (MMU).
- Optional Cryptographic Extension.
- Armv8.4 Dot Product instruction support.
- Superscalar, variable-length, out-of-order pipeline.
- Support for Arm TrustZone® technology.
- Support for *Page-Based Hardware Attributes* (PBHA).
- *Reliability, Availability, and Serviceability* (RAS) Extension.
- Full implementation of the Armv8.2-A A64, A32, and T32 instruction sets.
- *Generic Interrupt Controller* (GICv4) CPU interface to connect to an external distributor.
- Generic Timers interface supporting 64-bit count input from an external system counter.
- An integrated execution unit that implements the Advanced SIMD and floating-point architecture support.
- AArch32 Execution state at Exception level EL0 only. AArch64 Execution state at all Exception levels (EL0 to EL3).

How we checked auto vectorization?

- We checked the generated .s file and if it contains NEON registers like v0,v1,... then we can say that auto vectorization is being done.
- And with this auto vectorization, even we use only integer operations in our program we cannot say that FPU is sitting idle.

How performance was measured?

- Created large test inputs using c programs.
- Used clock_gettime to calculate precise time

Selected functions:

- Polynomial evaluation using Horner's method
- ncr calculation
- Cumulative summation of elements of a given array
- Binary search
- Factorial
- Prime number count in a given range of integers
- Matrix multiplication

Polynomial evaluation using Horner's method:

Version 1(Integer version):

```
clock_t start = clock();
long long result = coeff[0];
for (long long i = 1; i <= d; i++) {
    result = result * x + coeff[i];
}
clock_t end = clock();
```

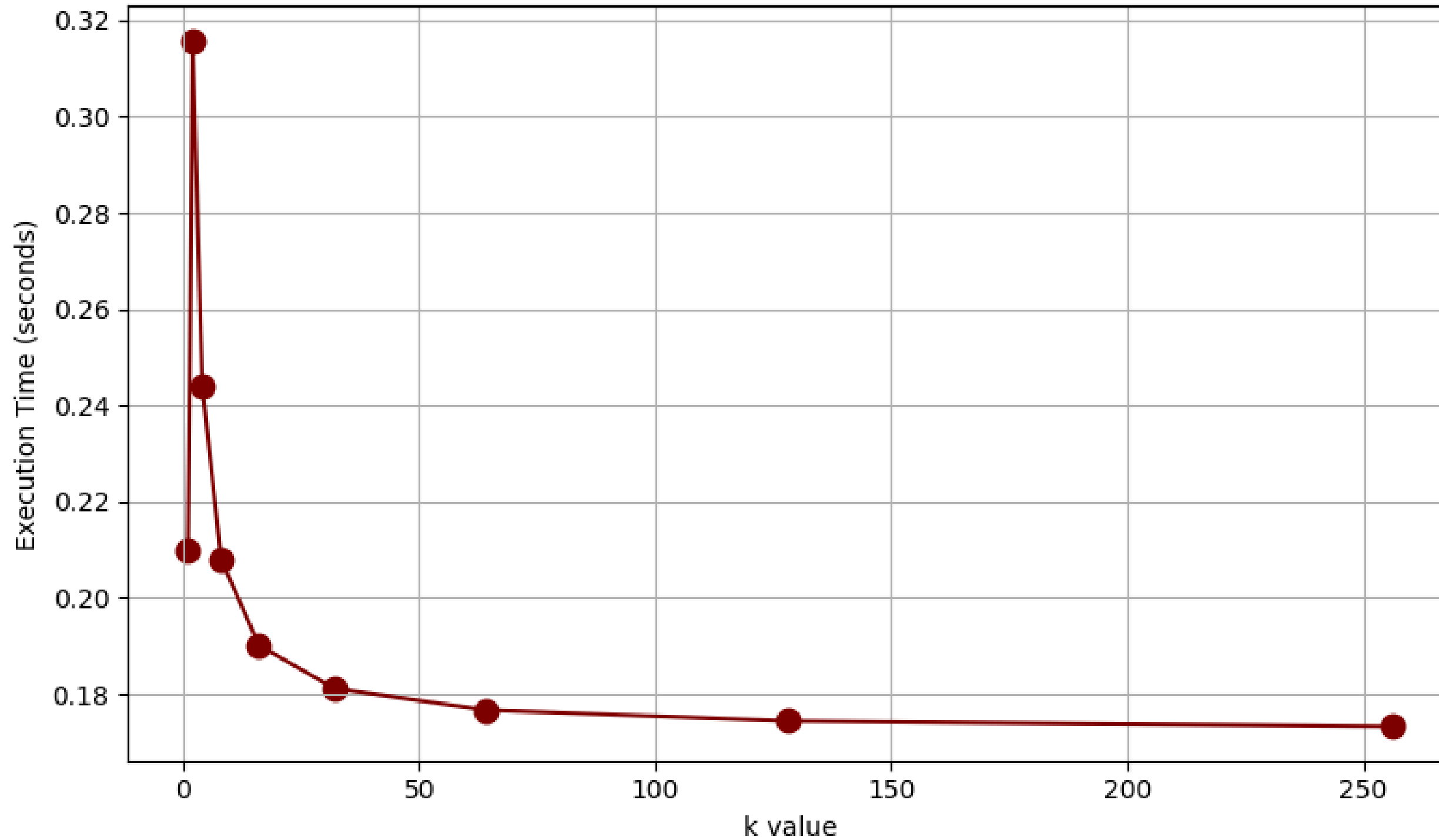
There is no auto vectorization here, so FPU is completely idle in the integer version.

Version 2 (k = 64):

```
long long mid = d / 64;
clock_t start1 = clock();
double r1=0.0;
for(long long i=d;i>=mid+1;i--){
    r1=fma(r1,x,coeff[i]);
}
double xp=pow(x,mid+1);
clock_t end1 = clock();
clock_t start2=clock();
long long r2=0;
for(long long i=mid;i>=0;i--){
    r2=r2*x+coeff[i];
}
double r2f=(double)r2;
double result=fma(r1,xp,r2);
clock_t end2=clock();
```

- **Degree is divided by k** to split the polynomial into independent parts.
- More work is assigned to the FPU to reduce ALU load from loop counters.
- ALU and FPU run in parallel during evaluation.

Polynomial Evaluation Time vs k



- When $k=2$ is taken the time actually increased
- The performance saturates at $k \geq 64$ since FPU is fully utilized.
- **Time improvement $\rightarrow$ 19.115%**

ncr calculation

version 1 (integer version):

```
double start = now();
for (long long int k = 0; k < repeats; k++){
    result = 1;
    dummy += 1;
    for (int i = 1; i <= r; i++) {
        result = result * (n - r + i) / i;
    }
}
double end=now();
```

version 2:

```
double start = now();
for (long long rep = 0; rep < repeats; rep++) {
    long long alu_tmp = 1;
    double fpu_tmp = 1.0;

    for (int i = 1; i <= fpuCount; i++) {
        if (i <= mid) {
            long long num = (n - r + i);
            alu_tmp = (alu_tmp * num) / (i);
        }

        fpu_tmp = (fpu_tmp * fpuNum[i-1]) / fpuDen[i-1];
    }

    result1 = alu_tmp;
    result2 = fpu_tmp;
    dummy++;
}
double end = now();
```

- even though the alu pipeline and fp pipeline overlap ,that does not reduce the time when compared to the integer version for all values of r.
- when r is small there are very few iterations .
- the loop executes sequentially.

Cumulative summation of an array

version 1(complete dependency):

```
double start_i = now();
for(int r=0; r < repeats; r++){
    for (int i = 0; i < m; i++)A[i] = B[i];
    for (int i = 1; i < m; i++)A[i] += A[i-1];
    dummy++;
}
double end_i = now();
```

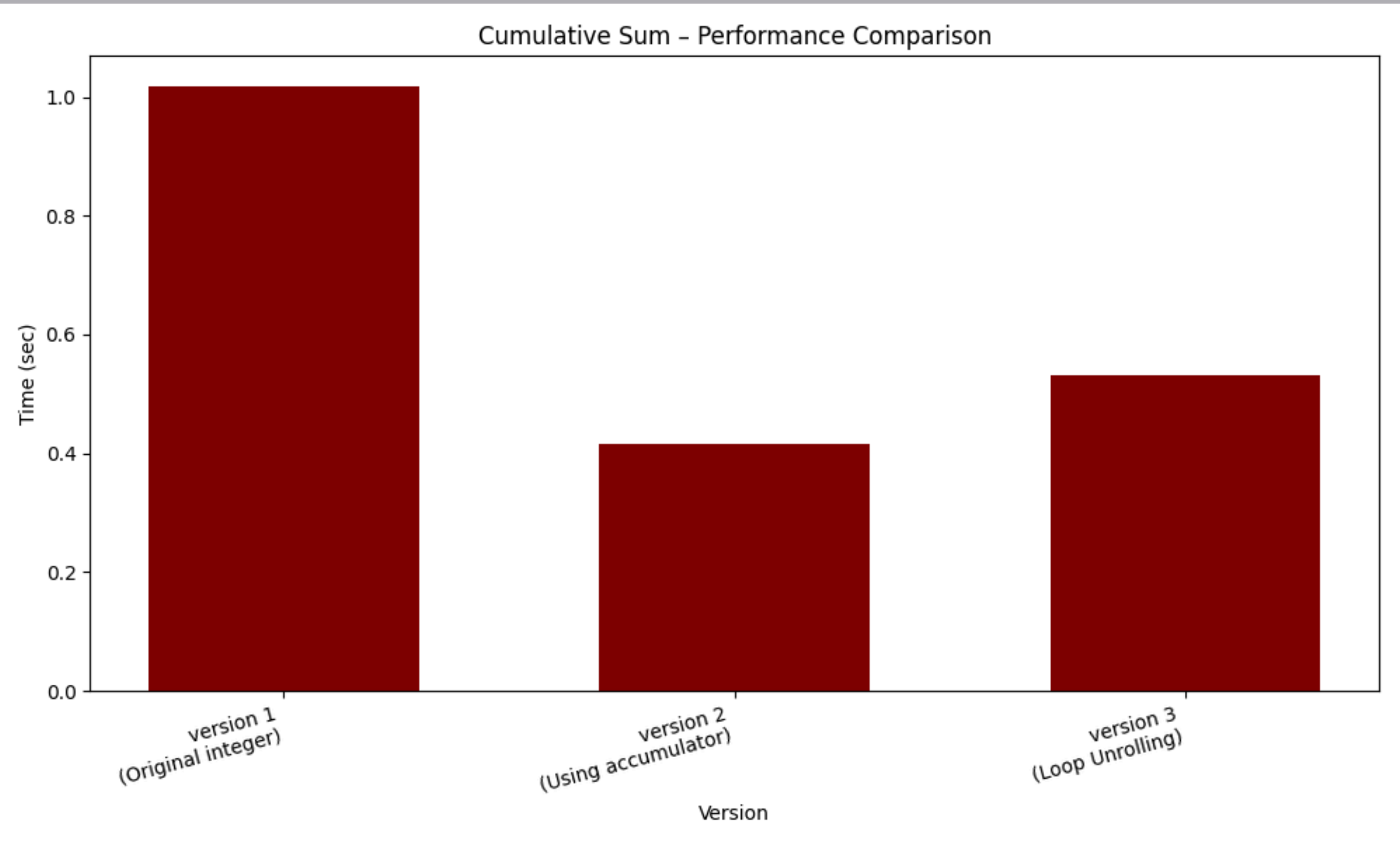
version 2:

```
double start_i = now();
for(int r=0; r < repeats; r++){
    sum=0;
    for (int i = 1; i < m; i++)sum += A[i];
    dummy++;
}
double end_i = now();
```

version 3(loop unrolling):

```
double start_i = now();
for(int r=0; r < repeats; r++){
    sum=0;
    fsum=0.0;
    for (int i = 1; i < m; i+=2){
        sum += A[i];
        fsum+= A[i-1];
    }
    dummy++;
}
double end_i = now();
```

- Accumulator form (version 2) was tried to remove the strong dependency and enable possible ALU–FPU parallelism.
- 3rd version does loop unrolling in order to run ALU and FPU in parallel.



- Time improvement for version 2 → 59.158%
- Time improvement for version 3 → 47.874%

- Accumulator was used to reduce the dependency chain so that the loop could be modified further (like for unrolling) and possibly allow more parallelism.
- Surprisingly, the accumulator version does 2 lane auto-vectorization, even though theoretically it should still have a dependency.
(Compiler likely treats integer addition as associative for optimization.)
- Loop unrolling was then tried on top of the accumulator version, but this stopped auto-vectorization
- Vectorization may have been avoided because reordering FP operations during unrolling can introduce rounding errors.

Binary search

```
double start = now();
for(long long int i=0 ;i<repeats;i++){
    l = 0;
    r = n - 1;
    ans = -1;
    dummy += 1;
    while (l <= r) {
        long long int mid = l + (r - l) / 2;

        if (arr[mid] == target) {
            ans = mid;
            break;
        } else if (arr[mid] < target) {
            l = mid + 1;
        } else {
            r = mid - 1;
        }
    }
}
double end = now();
```

- the loop is sequential
- it forms a strict dependency chain
- ALU is only used for mid computation and comparisons
- there are no independent instructions

Factorial

Integer version:

```
double start=now();
    for(int r=0;r<repeats;r++){
        p=1;
        for(int i=1;i<=n;i++){
            p*=i;
        }
        dummy++;
    }
double end=now();
```

2 way loop unrolling:

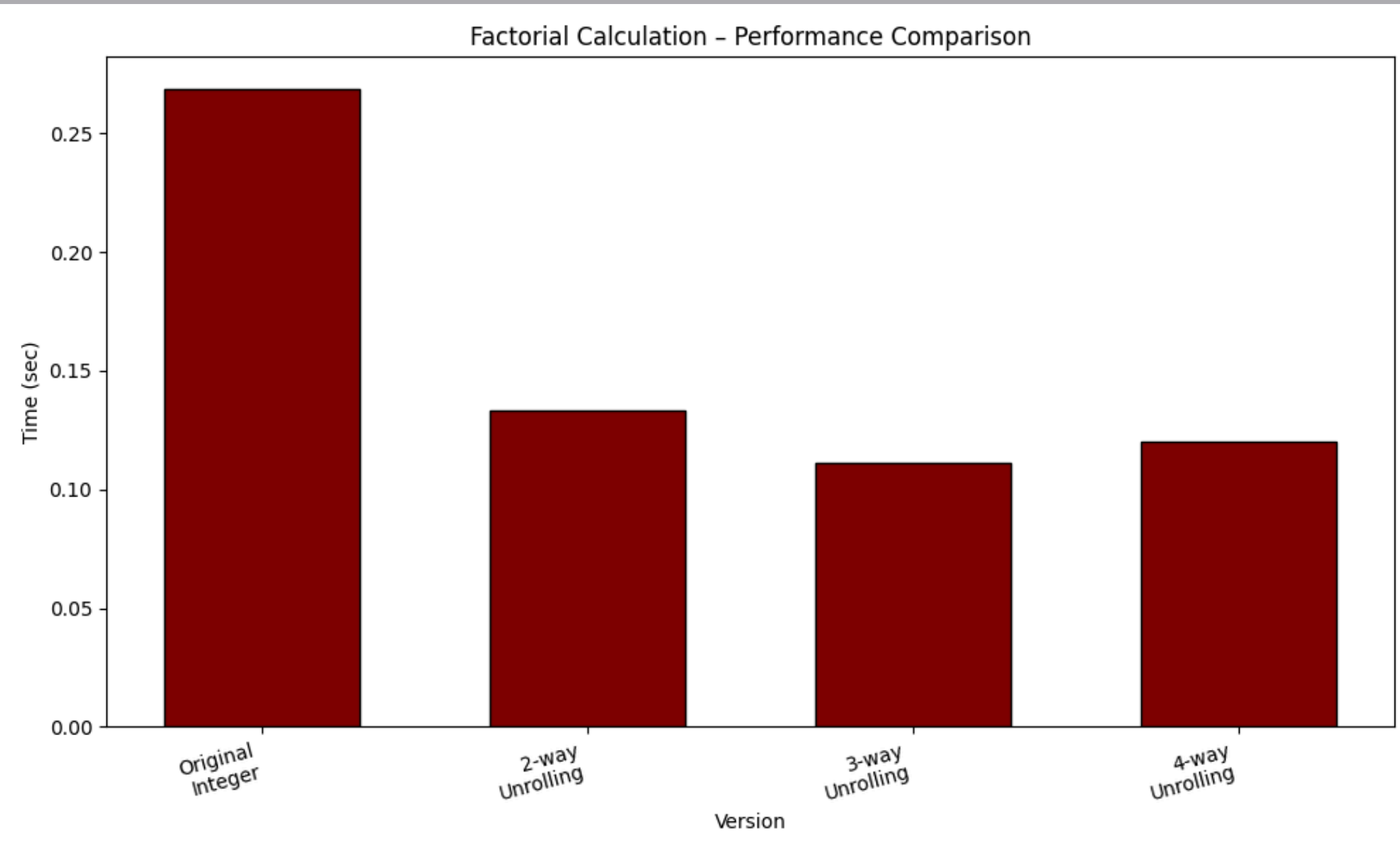
```
double start=now();
    for(int r=0;r<repeats;r++){
        p=1;
        pf=1.0;
        for(int i=2;i<=n;i+=2){
            p*=i;
            pf*=(i-1);
        }
        dummy++;
    }
    long long ans=p*(long long)pf;
    if(n%2==1) ans*=n;
double end=now();
```

3 way loop unrolling:

```
double start=now();
    for(int r=0;r<repeats;r++){
        p=1;
        pf1=1.0;
        pf2=1.0;
        for(int i=3;i<=n;i+=3){
            p*=i;
            pf1*=(i-1);
            pf2*=(i-2);
        }
        dummy++;
    }
    long long ans=p*(long long)pf1*(long long)pf2;
    if(n%3==1) ans*=n;
    else if(n%3==2) ans = ans*n*(n-1);
double end=now();
```

4 way loop unrolling:

```
double start=now();
    for(int r=0;r<repeats;r++){
        p1=1;
        p2=1;
        pf1=1.0;
        pf2=1.0;
        for(int i=4;i<=n;i+=4){
            p1*=i;
            p1*=(i-1);
            pf1*=(i-2);
            pf2*=(i-3);
        }
        dummy++;
    }
    long long ans=p1*p2*(long long)pf1*(long long)pf2;
    if(n%4==1) ans*=n;
    else if(n%4==2) ans = ans*n*(n-1);
    else if(n%4==3) ans=ans*n*(n-1)*(n-2);
double end=now();
```



- 2-way and 3-way unrolling balanced the work between ALU and FPU, allowing partial parallel progress
- 4-way unrolling became slightly slower because it overloaded the ALU (which also handles loop counters), increased register pressure

- Time improvement for 2 way loop unrolling → 50.427%
- 3 way loop unrolling → 58.564%
- 4 way loop unrolling → 55.251%

Counting prime numbers

version 1(only integer)

```
double begin = now();
for (long long num = start; num <= end; num++) {

    int isPrime = 1;

    if (num <= 1) {
        isPrime = 0;
    }
    else if (num <= 3) {
        isPrime = 1;
    }
    else if (num % 2 == 0 || num % 3 == 0) {
        isPrime = 0;
    }
    else {
        for (long long i = 5; i * i <= num; i += 6) {
            if (num % i == 0 || num % (i + 2) == 0) {
                isPrime = 0;
                break;
            }
        }
    }

    if (isPrime)
        count++;
}
double completed = now();
```

version 2

```
double begin = now();
while(num_int <= mid || num_double <= (double)end) {

    if(num_int <= mid) {
        int isPrime = 1;

        if(num_int <= 1) isPrime = 0;
        else if(num_int <= 3) isPrime = 1;
        else if(num_int % 2 == 0 || num_int % 3 == 0) isPrime = 0;
        else {
            for(long long i = 5; i*i <= num_int; i += 6) {
                if(num_int % i == 0 || num_int % (i+2) == 0) {
                    isPrime = 0;
                    break;
                }
            }
        }

        if(isPrime) count++;
        num_int++;
    }

    if(num_double <= (double)end) {
        int isPrimeF = 1;
        double nd = num_double;

        if(nd <= 1.0) isPrimeF = 0;
        else if(fp_mod(nd, 2.0) == 0.0 || fp_mod(nd, 3.0) == 0.0) isPrimeF = 0;
        else {
            for(double i = 5.0; i*i <= nd; i += 6.0) {
                if(fp_mod(nd, i) == 0.0 || fp_mod(nd, i+2.0) == 0.0) {
                    isPrimeF = 0;
                    break;
                }
            }
        }

        if(isPrimeF) count++;
        num_double += 1.0;
    }
}
double completed = now();
```

version 3 (loop unrolling)

```
double begin = now();
while(num <= end) {
    if(num <= end) {
        int isPrime = 1;

        if(num <= 1) isPrime = 0;
        else if(num <= 3) isPrime = 1;
        else if(num % 2 == 0 || num % 3 == 0) isPrime = 0;
        else {
            for(long long i = 5; i*i <= num; i += 6) {
                if(num % i == 0 || num % (i+2) == 0) {
                    isPrime = 0;
                    break;
                }
            }
        }

        if(isPrime) count++;
    }

    if(num + 1 <= end) {
        double nd = (double)(num + 1);
        int isPrimeF = 1;

        if(nd <= 1.0) isPrimeF = 0;
        else if(fp_mod(nd, 2.0) == 0.0 || fp_mod(nd, 3.0) == 0.0) isPrimeF = 0;
        else {
            for(double i = 5.0; i*i <= nd; i += 6.0) {
                if(fp_mod(nd, i) == 0.0 || fp_mod(nd, i+2.0) == 0.0) {
                    isPrimeF = 0;
                    break;
                }
            }
        }

        if(isPrimeF) count++;
    }

    num += 2;
}
double completed = now();
```

Matrix multiplication

- For large matrices, the compiler performs very aggressive auto-vectorization.
- So even pure integer code is already heavily using the FP/NEON unit.
- This leaves no room to add separate floating-point work in parallel.

Optimizations that can reduce time:

- Cache blocking (tiling)
- Manual SIMD vectorization
- Multi-core execution
- Combinations of above

Avoided since the scope is only ALU+FPU parallelism

General Guidelines to Use ALU & FPU in Parallel

- Split independent work so ALU handles integer parts while FPU handles floating-point parts.
- Shift more arithmetic to FPU to reduce ALU load (loop counters + address calculations use ALU).
- Break dependencies using loop unrolling or splitting (works only when operations don't depend on previous iteration).
- Avoid heavy int $\leftrightarrow$ float conversions as they increase number of instructions.
- Check for auto-vectorization—if SIMD is active, the FP/NEON unit is already busy and parallel ALU–FPU work may not help.

- This project clearly tests when the ALU and FPU can work in parallel on the Cortex-A76, using simple experiments

Major challenges faced:

- Time variation: OS noise, temperature, dynamic frequency.
- Data dependencies: Many loops were strictly sequential, limiting parallelism.
- Conversion overhead: int $\leftrightarrow$ float conversions adds costly delays.
- Unrolling correctness: Missing leftover terms or rounding errors had to be fixed

Limitations:

- Larger code size: Loop unrolling, splitting, and mixed ALU–FPU logic increase code complexity
- Precision issues: Float operations can introduce rounding errors
- Auto-vectorization issues: The compiler's own optimizations sometimes block or change the parallel work we try to add.
- May not work for tightly dependent loops.

Key learnings:

- ALU–FPU parallelism is not always possible
- Auto-vectorization affects the usage of floating operations
- Loop unrolling helps selectively

Contribution of each team member:

Each of us worked on different functions.

Amrutha

- Polynomial evaluation
- Cumulative summation of an array
- Factorial
- Matrix multiplication

Reethika

- ncr calculation
- binary search
- prime number count
- Matrix multiplication

THANK YOU!